

开源软件基础

单元测试

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

December 13, 2017



Outline

- 1 基本概念
- 2 unittest 模块
- 3 unittest.mock

软件测试

软件测试（英语：software testing），描述一种用来促进鉴定软件的正确性、完整性、安全性和质量的过程。据此，您可能会想，软件测试永远不可能完整的确立任意电脑软件的正确性。然而，在可计算理论（计算机科学的一个支派）一个简单的数学证明推断出下列结果：不可能完全解决所谓“死机”，指任意计算机程序是否会进入死循环，或者罢工并产生输出问题。换句话说，软件测试是一种实际输出与预期输出间的审核或者比较过程。

软件测试的经典定义是：在规定的条件下对程序进行操作，以发现程序错误，衡量软件质量，并对其是否能满足设计要求进行评估的过程。¹

¹<https://zh.wikipedia.org/zh-cn/>

测试的阶段

测试的阶段

单元测试

单元测试是对软件组成单元进行测试，其目的是检验软件基本组成单位的正确性，测试的对象是软件设计的最小单位：函数。

测试的阶段

集成测试

集成测试也称综合测试、组装测试、联合测试，将程序模块采用适当的集成策略组装起来，对系统的接口及集成后的功能进行正确性检测的测试工作。其主要目的是检查软件单位之间的接口是否正确，集成测试的对象是已经经过单元测试的模块。

测试的阶段

系统测试

系统测试主要包括功能测试、界面测试、可靠性测试、易用性测试、性能测试。功能测试主要针对包括功能可用性、功能实现程度（功能流程 & 业务流程、数据处理 & 业务数据处理）方面测试。

测试的阶段

回归测试

回归测试指在软件维护阶段，为了检测代码修改而引入的错误所进行的测试活动。回归测试是软件维护阶段的重要工作，有研究表明，回归测试带来的耗费占软件生命周期的 1/3 总费用以上。

与普通的测试不同，在回归测试过程开始的时候，测试者有一个完整的测试用例集可供使用，因此，如何根据代码的修改情况对已有测试用例集进行有效的复用是回归测试研究的重要方向，此外，回归测试的研究方向还涉及自动化工具，面向对象回归测试，测试用例优先级，回归测试用例补充生成等。

单元测试

在计算机编程中，单元测试（英语：Unit Testing）又称为模块测试，是针对程序模块（软件设计的最小单位）来进行正确性检验的测试工作。程序单元是应用的最小可测试部件。在过程化编程中，一个单元就是单个程序、函数、过程等；对于面向对象编程，最小单元就是方法，包括基类（超类）、抽象类、或者派生类（子类）中的方法。

通常来说，程序员每修改一次程序就会进行最少一次单元测试，在编写程序的过程中前后很可能要进行多次单元测试，以证实程序达到软件规格书要求的工作目标，没有程序错误；虽然单元测试不是什么必须的，但也不坏，这牵涉到项目管理的政策决定。

每个理想的测试案例独立于其它案例；为测试时隔离模块，经常使用 stubs、mock 或 fake 等测试马甲程序。单元测试通常由软件开发人员编写，用于确保他们所写的代码匹配软件需求和遵循开发目标。它的实施方式可以是非常手动的（通过纸笔），或者是做成构建自动化的一部分。

2

²<https://zh.wikipedia.org/zh-cn/>

unittest — Unit testing framework

The unittest unit testing framework was originally inspired by JUnit and has a similar flavor as major unit testing frameworks in other languages. It supports test automation, sharing of setup and shutdown code for tests, aggregation of tests into collections, and independence of the tests from the reporting framework.

Outline

- 1 基本概念
- 2 **unittest 模块**
- 3 unittest.mock

Basic example

Here is a short script to test three string methods:

Basic example

```
import unittest

class TestStringMethods(unittest.TestCase):

    def test_upper(self):
        self.assertEqual('foo'.upper(), 'FOO')

    def test_isupper(self):
        self.assertTrue('FOO'.isupper())
        self.assertFalse('Foo'.isupper())

    def test_split(self):
        s = 'hello world'
```

Basic example

A testcase is created by subclassing `unittest.TestCase`. The three individual tests are defined with methods whose names start with the letters `test`. This naming convention informs the test runner about which methods represent tests.

Basic example

The crux of each test is a call to `TestCase.assertEqual` to check for an expected result; `TestCase.assertTrue` or `TestCase.assertFalse` to verify a condition; or `TestCase.assertRaises` to verify that a specific exception gets raised. These methods are used instead of the `assert` statement so the test runner can accumulate all test results and produce a report.

Basic example

The `TestCase.setUp` and `TestCase.tearDown` methods allow you to define instructions that will be executed before and after each test method. They are covered in more detail in the section `organizing-tests`.

Basic example

The final block shows a simple way to run the tests. `unittest.main` provides a command-line interface to the test script. When run from the command line, the above script produces an output that looks like this:

Basic example

...

Ran 3 tests in 0.000s

OK

Basic example

Passing the `-v` option to your test script will instruct `unittest.main` to enable a higher level of verbosity, and produce the following output:

Basic example

```
test_isupper (__main__.TestStringMethods) ... ok  
test_split (__main__.TestStringMethods) ... ok  
test_upper (__main__.TestStringMethods) ... ok
```

Ran 3 tests in 0.001s

OK

Basic example

The above examples show the most commonly used unittest features which are sufficient to meet many everyday testing needs. The remainder of the documentation explores the full feature set from first principles.

Command-Line Interface

The unittest module can be used from the command line to run tests from modules, classes or even individual test methods:

Command-Line Interface

```
python -m unittest test_module1 test_module2  
python -m unittest test_module.TestClass  
python -m unittest test_module.TestClass.test_method
```

Command-Line Interface

You can pass in a list with any combination of module names, and fully qualified class or method names.

Command-Line Interface

Test modules can be specified by file path as well:

Command-Line Interface

```
python -m unittest tests/test_something.py
```

Command-Line Interface

This allows you to use the shell filename completion to specify the test module. The file specified must still be importable as a module. The path is converted to a module name by removing the `.py` and converting path separators into `..`. If you want to execute a test file that isn't importable as a module you should execute the file directly instead.

Command-Line Interface

You can run tests with more detail (higher verbosity) by passing in the `-v` flag:

Command-Line Interface

```
python -m unittest -v test_module
```

Command-Line Interface

When executed without arguments unittest-test-discovery is started:

Command-Line Interface

```
python -m unittest
```

Command-Line Interface

For a list of all the command-line options:

Command-Line Interface

```
python -m unittest -h
```

Outline

- 1 基本概念
- 2 unittest 模块
- 3 **unittest.mock**

Command-line options

unittest supports these command-line options:

Command-line options

The command line can also be used for test discovery, for running all of the tests in a project or just a subset.

Test Discovery

Unittest supports simple test discovery. In order to be compatible with test discovery, all of the test files must be modules <tut-modules> or packages <tut-packages> (including :term:‘namespace packages <namespace package>‘) importable from the top-level directory of the project (this means that their filenames must be valid :ref:‘identifiers <identifiers>‘).

Test Discovery

Test discovery is implemented in `TestLoader.discover`, but can also be used from the command line. The basic command-line usage is:

Test Discovery

```
cd project_directory  
python -m unittest discover
```

Test Discovery

Note

Test Discovery

As a shortcut, `python -m unittest` is the equivalent of `python -m unittest discover`. If you want to pass arguments to test discovery the `discover` sub-command must be used explicitly.

Test Discovery

The discover sub-command has the following options:

Test Discovery

The -s, -p, and -t options can be passed in as positional arguments in that order. The following two command lines are equivalent:

Test Discovery

```
python -m unittest discover -s project_directory -p "*  
python -m unittest discover project_directory "*_test.
```

Test Discovery

As well as being a path it is possible to pass a package name, for example `myproject.subpackage.test`, as the start directory. The package name you supply will then be imported and its location on the filesystem will be used as the start directory.

Test Discovery

Caution!

Test Discovery

Test discovery loads tests by importing them. Once test discovery has found all the test files from the start directory you specify it turns the paths into package names to import. For example `foo/bar/baz.py` will be imported as `foo.bar.baz`.

Test Discovery

If you have a package installed globally and attempt test discovery on a different copy of the package then the import could happen from the wrong place. If this happens test discovery will warn you and exit.

Test Discovery

If you supply the start directory as a package name rather than a path to a directory then discover assumes that whichever location it imports from is the location you intended, so you will not get the warning.

Test Discovery

Test modules and packages can customize test loading and discovery by through the `load_tests` protocol.

Organizing test code

The basic building blocks of unit testing are test cases — single scenarios that must be set up and checked for correctness. In `unittest`, test cases are represented by `unittest.TestCase` instances. To make your own test cases you must write subclasses of `TestCase` or use `FunctionTestCase`.

Organizing test code

The testing code of a `TestCase` instance should be entirely self contained, such that it can be run either in isolation or in arbitrary combination with any number of other test cases.

Organizing test code

The simplest TestCase subclass will simply implement a test method (i.e. a method whose name starts with test) in order to perform specific testing code:

Organizing test code

```
import unittest

class DefaultWidgetSizeTestCase(unittest.TestCase):
    def test_default_widget_size(self):
        widget = Widget('The widget')
        self.assertEqual(widget.size(), (50, 50))
```

Organizing test code

Note that in order to test something, we use one of the `assert*` methods provided by the `TestCase` base class. If the test fails, an exception will be raised, and `unittest` will identify the test case as a failure. Any other exceptions will be treated as errors.

Organizing test code

Tests can be numerous, and their set-up can be repetitive. Luckily, we can factor out set-up code by implementing a method called `TestCase.setUp`, which the testing framework will automatically call for every single test we run:

Organizing test code

```
import unittest

class WidgetTestCase(unittest.TestCase):
    def setUp(self):
        self.widget = Widget('The widget')

    def test_default_widget_size(self):
        self.assertEqual(self.widget.size(), (50, 50),
                          'incorrect default size')

    def test_widget_resize(self):
        self.widget.resize(100, 150)
        self.assertEqual(self.widget.size(), (100, 150))
```



Organizing test code

Note

Organizing test code

The order in which the various tests will be run is determined by sorting the test method names with respect to the built-in ordering for strings.

Organizing test code

If the `TestCase.setUp` method raises an exception while the test is running, the framework will consider the test to have suffered an error, and the test method will not be executed.

Organizing test code

Similarly, we can provide a `TestCase.tearDown` method that tidies up after the test method has been run:

Organizing test code

```
import unittest

class WidgetTestCase(unittest.TestCase):
    def setUp(self):
        self.widget = Widget('The widget')

    def tearDown(self):
        self.widget.dispose()
```

Organizing test code

If `TestCase.setUp` succeeded, `TestCase.tearDown` will be run whether the test method succeeded or not.

Organizing test code

Such a working environment for the testing code is called a fixture.

Organizing test code

Test case instances are grouped together according to the features they test. unittest provides a mechanism for this test suite, represented by unittest's `TestSuite` class. In most cases, calling `unittest.main` will do the right thing and collect all the module's test cases for you, and then execute them.

Organizing test code

However, should you want to customize the building of your test suite, you can do it yourself:

Organizing test code

```
def suite():  
    suite = unittest.TestSuite()  
    suite.addTest(WidgetTestCase('test_default_size'))  
    suite.addTest(WidgetTestCase('test_resize'))  
    return suite
```

Organizing test code

You can place the definitions of test cases and test suites in the same modules as the code they are to test (such as `widget.py`), but there are several advantages to placing the test code in a separate module, such as `test_widget.py`:

Re-using old test code

Some users will find that they have existing test code that they would like to run from unittest, without converting every old test function to a TestCase subclass.

Re-using old test code

For this reason, unittest provides a `FunctionTestCase` class. This subclass of `TestCase` can be used to wrap an existing test function. Set-up and tear-down functions can also be provided.

Re-using old test code

Given the following test function:

Re-using old test code

```
def testSomething():  
    something = makeSomething()  
    assert something.name is not None  
    # ...
```


Re-using old test code

one can create an equivalent test case instance as follows, with optional set-up and tear-down methods:

Re-using old test code

```
testcase = unittest.FunctionTestCase(testSomething,  
                                     setUp=makeSomething,  
                                     tearDown=deleteSomething)
```

Re-using old test code

Note

Re-using old test code

Even though `FunctionTestCase` can be used to quickly convert an existing test base over to a unittest-based system, this approach is not recommended. Taking the time to set up proper `TestCase` subclasses will make future test refactorings infinitely easier.

Re-using old test code

In some cases, the existing tests may have been written using the doctest module. If so, doctest provides a DocTestSuite class that can automatically build unittest.TestSuite instances from the existing doctest-based tests.

Skipping tests and expected failures

Unittest supports skipping individual test methods and even whole classes of tests. In addition, it supports marking a test as an "expected failure," a test that is broken and will fail, but shouldn't be counted as a failure on a `TestResult`.

Skipping tests and expected failures

Skipping a test is simply a matter of using the skip decorator or one of its conditional variants.

Skipping tests and expected failures

Basic skipping looks like this:

Skipping tests and expected failures

```
class MyTestCase(unittest.TestCase):

    @unittest.skip("demonstrating skipping")
    def test_nothing(self):
        self.fail("shouldn't happen")

    @unittest.skipIf(mylib.__version__ < (1, 3),
                    "not supported in this library version")
    def test_format(self):
        # Tests that work for only a certain version of mylib
        pass

    @unittest.skipUnless(sys.platform.startswith("win"),
```



Skipping tests and expected failures

This is the output of running the example above in verbose mode:

Skipping tests and expected failures

```
test_format (__main__.MyTestCase) ... skipped 'not supported'
test_nothing (__main__.MyTestCase) ... skipped 'demonstration'
test_windows_support (__main__.MyTestCase) ... skipped 'not supported'
```

```
Ran 3 tests in 0.005s
```

```
OK (skipped=3)
```

Skipping tests and expected failures

Classes can be skipped just like methods:

Skipping tests and expected failures

```
@unittest.skip("showing class skipping")  
class MySkippedTestCase(unittest.TestCase):  
    def test_not_run(self):  
        pass
```

Skipping tests and expected failures

`TestCase.setUp` can also skip the test. This is useful when a resource that needs to be set up is not available.

Skipping tests and expected failures

Expected failures use the `expectedFailure` decorator.

Skipping tests and expected failures

```
class ExpectedFailureTestCase(unittest.TestCase):  
    @unittest.expectedFailure  
    def test_fail(self):  
        self.assertEqual(1, 0, "broken")
```


Skipping tests and expected failures

It's easy to roll your own skipping decorators by making a decorator that calls skip on the test when it wants it to be skipped. This decorator skips the test unless the passed object has a certain attribute:

Skipping tests and expected failures

```
def skipUnlessHasattr(obj, attr):  
    if hasattr(obj, attr):  
        return lambda func: func  
    return unittest.skip("{!r} doesn't have {!r}".format
```

Skipping tests and expected failures

The following decorators implement test skipping and expected failures:

Skipping tests and expected failures

Skipped tests will not have `TestCase.setUp` or `TestCase.tearDown` run around them. Skipped classes will not have `TestCase.setUpClass` or `TestCase.tearDownClass` run. Skipped modules will not have `setUpModule` or `tearDownModule` run.

Distinguishing test iterations using subtests

When some of your tests differ only by a some very small differences, for instance some parameters, unittest allows you to distinguish them inside the body of a test method using the `TestCase.subTest` context manager.

Distinguishing test iterations using subtests

For example, the following test:

Distinguishing test iterations using subtests

```
class NumbersTest(unittest.TestCase):

    def test_even(self):
        """
        Test that numbers between 0 and 5 are all even
        """
        for i in range(0, 6):
            with self.subTest(i=i):
                self.assertEqual(i % 2, 0)
```

Distinguishing test iterations using subtests

will produce the following output:

Distinguishing test iterations using subtests

```
=====
FAIL: test_even (__main__.NumbersTest) (i=1)
-----
```

```
Traceback (most recent call last):
```

```
  File "subtests.py", line 32, in test_even
    self.assertEqual(i % 2, 0)
```

```
AssertionError: 1 != 0
```

```
=====
FAIL: test_even (__main__.NumbersTest) (i=3)
-----
```

```
Traceback (most recent call last):
```

```
  File "subtests.py", line 32, in test_even
```



Distinguishing test iterations using subtests

Without using a subtest, execution would stop after the first failure, and the error would be less easy to diagnose because the value of `i` wouldn't be displayed:

Distinguishing test iterations using subtests

```
=====
FAIL: test_even (__main__.NumbersTest)
-----
```

```
Traceback (most recent call last):
```

```
  File "subtests.py", line 32, in test_even
```

```
    self.assertEqual(i % 2, 0)
```

```
AssertionError: 1 != 0
```

Classes and functions

This section describes in depth the API of unittest.

Outline

- 1 基本概念
- 2 unittest 模块
- 3 unittest.mock

Test cases

Instances of the `TestCase` class represent the logical test units in the unittest universe. This class is intended to be used as a base class, with specific tests being implemented by concrete subclasses. This class implements the interface needed by the test runner to allow it to drive the tests, and methods that the test code can use to check for and report various kinds of failure.

Test cases

Each instance of `TestCase` will run a single base method: the method named `methodName`. In most uses of `TestCase`, you will neither change the `methodName` nor reimplement the default `runTest()` method.

Test cases

TestCase instances provide three groups of methods: one group used to run the test, another used by the test implementation to check conditions and report failures, and some inquiry methods allowing information about the test itself to be gathered.

Test cases

Methods in the first group (running the test) are:

Test cases

The TestCase class provides several assert methods to check for and report failures. The following table lists the most commonly used methods (see the tables below for more assert methods):

Test cases

All the assert methods accept a `msg` argument that, if specified, is used as the error message on failure (see also `longMessage`). Note that the `msg` keyword argument can be passed to `assertRaises`, `assertRaisesRegex`, `assertWarns`, `assertWarnsRegex` only when they are used as a context manager.

Test cases

It is also possible to check the production of exceptions, warnings, and log messages using the following methods:

Test cases

There are also other methods used to perform more specific checks, such as:

Test cases

The `assertEqual` method dispatches the equality check for objects of the same type to different type-specific methods. These methods are already implemented for most of the built-in types, but it's also possible to register new methods using `addTypeEqualityFunc`:

Test cases

The list of type-specific methods automatically used by `TestCase.assertEqual` are summarized in the following table. Note that it's usually not necessary to invoke these methods directly.

Test cases

Finally the TestCase provides the following methods and attributes:

Test cases

Testing frameworks can use the following methods to collect information on the test:

Test cases

This class implements the portion of the TestCase interface which allows the test runner to drive the test, but does not provide the methods which test code can use to check and report errors. This is used to create test cases using legacy test code, allowing it to be integrated into a unittest-based test framework.

Test cases

For historical reasons, some of the TestCase methods had one or more aliases that are now deprecated. The following table lists the correct names along with their deprecated aliases:

Outline

- 1 基本概念
- 2 unittest 模块
- 3 unittest.mock